Ders 7 Çalışma Özeti

Ders 7 Çalışma Özeti

Genel Konular

- Biyolojik nörondan yapay nörona geçiş (soma, dendrit, akson, sinaps).
- Tek katmanlı perceptron (single-layer perceptron) modeli ve doğrusal (lineer) ayırma.
- Aktivasyon fonksiyonları (activation function): step, sign, sigmoid, lineer.
- Hata (cost) fonksiyonu, gradient descent ve delta kuralı ile ağırlık güncelleme.
- Öğrenme oranı (learning rate), lokal ve global minimum.
- Perceptron öğrenme algoritması: AND ve OR için adım adım örnekler.
- Epoch (ipok) ile iterasyon farkı.
- XOR problemi ve tek katmanlı perceptronun sınırı.
- Çok katmanlı perceptron (multi-layer perceptron, MLP) ve geri yayılım (backpropagation).
- Momentum ve hiperbolik tanjant ile daha hızlı öğrenme.
- Vize ve ödev duyuruları.

Hocanın Özellikle Vurguladığı Kısımlar

- Öğrenme tamamen ağırlıkların değişmesidir. Girişler ve istenen çıkışlar sabittir; değişebilen tek şey ağırlıklardır.
- Eşik seviyesi (threshold) ile bias aynı şeydir. Eşik ayrı bir θ olarak da, x_0 girişine bağlı bir w_0 ağırlığı olarak da yazılabilir. Dokümanlarda farklı gösterimler görürsünüz; matematiksel olarak hepsi aynı kapıya çıkar.
- "Büyüktür" mü "büyük eşittir" mi kullanıldığına takılmayın. Toplam tam eşiğe denk gelirse bir sonraki ağırlık değişiminde zaten bir tarafa kayar.
- Perceptron yalnız doğrusal ayrılabilen (linearly separable) sınıfları ayırabilir. XOR bu yüzden tek katmanla çözülemez.
- Learning rate büyükse global minimumu atlayıp geçebilirsiniz; küçükse yavaş öğrenirsiniz. Başta büyük alıp iterasyonlar ilerledikçe küçültmek iyi bir yoldur.
- Backpropagation adını hatanın çıkış katmanından giriş katmanına doğru katman katman geri yayılmasından alır. Önce çıkışa giden ağırlıklar, sonra gizli katmana giden ağırlıklar güncellenir.
- Çıkış nöronu sayısı sınıf sayısına bağlıdır. 0-9 rakamları için 10 çıkış nöronu kullanılabilir; ikili kodlama ile 4 nöron da yeter ($2^4 = 16 \geq 10$).
- Hoca XOR backpropagation örneğinin bulunduğu slaytları özellikle paylaşacağını söyledi: "Bir tarafa elle çizin, bir tarafa slaytları alın ve y_1, y_2, y_3'ün nasıl hesaplandığını tek tek görün."

Kısa Tekrar Notları

- Perceptron çıkışı: $y = \text{sign}\left(\sum_{i=1}^{n} w_i x_i - \theta\right)$.
- Hata: $e(p) = y_d(p) - y(p)$ (istenen çıkış eksi hesaplanan çıkış).
- Delta kuralı: $\Delta w_i(p) = \alpha \cdot x_i(p) \cdot e(p),\ \ w_i(p+1) = w_i(p) + \Delta w_i(p)$.
- $e > 0$ ise ağırlık artar, çıkış hedefe doğru büyür. $e < 0$ ise ağırlık azalır.
- Learning rate α 0 ile 1 arasındadır. 0'a yakın yavaş, 1'e yakın hızlı öğrenme demektir.
- Epoch: eğitim kümesindeki tüm örneklerin bir kez gösterilmesi. İterasyon: tek bir örneğin gösterilmesi.

- Öğrenme, bir epoch boyunca hiç hata kalmayınca (veya toplam hata belirlenen eşiğin altına inince) biter.
- Sigmoid: $y = \dfrac{1}{1+e^{-x}}$. Yumuşak geçiş sağlar; MLP'de kullanılır.
- MLP'de ileri yönde çıkış hesaplanır, geri yönde hata düzeltilir.
- Momentum (genelde $\beta \approx 0.9$) öğrenmeyi hızlandırır ve daha büyük α kullanılmasına izin verir.

Detaylı Açıklamalar

1. Biyolojik Nörondan Yapay Nörona

- Yapay zekanın hedefi insanın beş duyuyla yaptığı işleri (görme, duyma, anlama) bilgisayara yaptırmaktır. Örnekler: parmak izi tanıma, yüz tanıma, ELIZA adlı "psikolog" program.
- İlk sistemler matematiğe dayalıydı ama insan gibi davranmıyordu. İnsanın asıl özelliği öğrenebilmesidir. Bu yüzden öğrenme yeteneği olan sistemler geliştirme yolu açıldı.
- İnsan beyninde yaklaşık 10^{11} nöron vardır. Her nöron yaklaşık 10^4 nörona bağlıdır.
- Nörona gelen sinyaller ağırlıklı toplanır. Toplam belli bir eşiğin üzerindeyse aksona pozitif sinyal gider, değilse hücre bir işlem yapmaz.
- Beyin paralel ve dağıtık çalışan bir sistemdir. Örneğin ders dinlerken aynı anda not alabilir, üşüdüğünüzü fark edebilirsiniz.
- Yapay nöron çalışmaları 1940'ların sonunda başladı. Donanım yetersiz olduğu için 2000'lerin başına kadar çok basit modeller kullanıldı.
- Donanımlar güçlenince ve büyük veri ortaya çıkınca derin öğrenme (deep learning) doğdu. Mantık: çok örnek görmek genellemeyi kolaylaştırır. Hoca örnek olarak her kadına "anne" diyen küçük çocuğun zamanla ayırt etmeyi öğrenmesini verdi.

Biyolojik ve yapay nöron karşılıkları

Biyolojik nöron	Yapay nöron
Soma (hücre gövdesi)	Nöron (toplayıcı)
Dendrit	Giriş (input)
Akson	Çıkış (output)
Sinaps	Ağırlık (weight)

2. Perceptron Modeli

- Yapay nöron mantığı McCulloch ve Pitts tarafından önerildi (altyazıda "Mark Klok ve Pitts"). Perceptronun öğrenme ile ilk gösterimi Rosenblatt'a aittir (1958).
- Giriş $\mathbf{x} = (x_1, x_2, \dots, x_n)$ bir özellik vektörüdür. Örnek: bir kişinin yaşı, boyu, kilosu; bir ses sinyali; bir görüntü.
- Her özelliğin etkisi farklıdır. Bu etkiyi ağırlıklar belirler.
- Ağırlıklı toplam iki vektörün iç çarpımıdır (dot product):

$$\sum_{i=1}^{n} w_i x_i = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n$$

- Toplam bir aktivasyon fonksiyonundan geçirilir. Sign fonksiyonu kullanılırsa:

$$y = \begin{cases} +1, & \sum_i w_i x_i \geq \theta \\ -1, & \sum_i w_i x_i < \theta \end{cases}$$

- $+1$ A sınıfı, -1 B sınıfı olarak yorumlanır. Böylece iki sınıflı bir sınıflandırıcı elde edilir.
- İki özellik için $w_1 x_1 + w_2 x_2 - \theta = 0$ doğrusu iki sınıfı ayırır. Bu sınıra hiper düzlem (hyperplane) ya da karar sınırı (decision boundary) denir. Sınırın üstü $y = 1$, altı $y = -1$ bölgesidir.

- Bir öğrenci hiper düzlemin orijinden geçmesi gerekip gerekmediğini sordu. Hoca: gerekmez.

Eşiğin üç farklı yazımı

- $\sum_{i=1}^{n} w_i x_i - \theta$ ile karşılaştırma.
- $\sum_{i=0}^{n} w_i x_i$, burada $x_0 = -1$ (veya $+1$) ve $w_0 = \theta$. Kod yazarken kolaylık sağlar. x_0 gerçek bir özellik değildir.
- $\sum_{i=1}^{n} w_i x_i > \theta$ koşulu.
- Bazı kaynaklarda eşik sabit tutulur, yalnız ağırlıklar değişir. Bazılarında eşik de ağırlık gibi güncellenir.

3. Aktivasyon Fonksiyonları

- **Step:** $y = 1$ $(x \geq 0)$, $y = 0$ $(x < 0)$.
- **Sign:** $y = +1$ $(x \geq 0)$, $y = -1$ $(x < 0)$. Ani geçiş sağlar.
- **Sigmoid:** $y = \dfrac{1}{1 + e^{-x}}$. S harfine benzer, yumuşak geçiş sağlar. Lojistik regresyonda da kullanılmıştı.
- **Lineer:** çıkış girişle doğrusal orantılı. Çok kullanılmaz.
- Tek katmanlı perceptronda genelde step veya sign kullanılır. MLP'de sigmoid kullanılır.

4. Hata Fonksiyonu ve Gradient Descent

- Maliyet (cost) fonksiyonu hesaplanan çıkışın istenen çıkışa ne kadar yakın olduğunu ölçer. Hatanın karesi alınır (sum of squared errors). Kare alma hatayı pozitif yapar ve büyük hataları büyütür. Başa konan $\frac{1}{2}$ türevi kolaylaştırır.

$$E = \frac{1}{2} \sum_{j} \left(y_{d,j} - y_j \right)^2$$

- Ağırlığa göre türev alınınca ağırlığın nasıl değişmesi gerektiği bulunur: $(y_d - y)\, x_i$.
- Ağırlıklar başta rastgele seçilir. Hata yüzeyinde yavaş yavaş inilerek global minimuma ulaşılmaya çalışılır (gradient descent).
- Delta kuralı (Widrow-Hoff kuralı; altyazıda "Hofloin"):

$$w_i(t+1) = w_i(t) + \alpha\, (y_d - y)\, x_i$$

- Hata yüzeyi düzgün bir eğri değildir. İki tehlike vardır: lokal minimuma takılmak ve büyük adımla global minimumu atlamak.
- Örnek: $\alpha = 0.1$ ise ağırlık yavaş değişir, $\alpha = 0.9$ ise hızlı değişir. Strateji: başta büyük α, sonra küçültmek.

5. Perceptron Öğrenme Algoritması

1. **Başlatma (initialization):** ağırlıkları ve eşiği genelde $[-0.5, +0.5]$ aralığında rastgele değerlerle başlat.
2. **Aktivasyon:** t anındaki çıkışı hesapla: $y(t) = \text{step}\left(\sum_i x_i w_i - \theta\right)$.
3. **Ağırlık eğitimi (weight training):** $e(t) = y_d(t) - y(t)$, $\Delta w_i(t) = \alpha\, x_i\, e(t)$, $w_i(t+1) = w_i(t) + \Delta w_i(t)$.
4. **İterasyon:** 2. adıma dön. Yakınsama (convergence) olana, yani hata eşiğin altına inene kadar sürdür.

Örnek 1: AND öğrenen perceptron (slayttaki tablo)

- Başlangıç: $w_1 = 0.3$, $w_2 = -0.1$, $\theta = 0.2$ (sabit), $\alpha = 0.1$, step fonksiyonu.
- Not: Hoca anlatırken w_2'yi önce 0.1 yazdı, sonra "w_2 eksi 0,1'miş" diye düzeltti.
- 1. epoch:

- $(0,0)$: toplam $0 < 0.2 \Rightarrow y = 0$, $y_d = 0$, $e = 0$. Ağırlıklar değişmez.
- $(0,1)$: toplam $-0.1 < 0.2 \Rightarrow y = 0$, $e = 0$. Değişmez.
- $(1,0)$: toplam $0.3 > 0.2 \Rightarrow y = 1$, $y_d = 0$, $e = -1$. $\Delta w_1 = 0.1 \cdot (-1) \cdot 1 = -0.1 \Rightarrow w_1 = 0.2$. $x_2 = 0$ olduğu için $w_2 = -0.1$ kalır.
- $(1,1)$: toplam $0.2 - 0.1 = 0.1 < 0.2 \Rightarrow y = 0$, $y_d = 1$, $e = 1$. $w_1 = 0.2 + 0.1 = 0.3$, $w_2 = -0.1 + 0.1 = 0.0$.

- Ağırlıklar epoch'lar boyunca böyle değişir. Slayttaki tabloda 5. epoch'ta hiç hata kalmaz. Son ağırlıklar $w_1 = 0.1$, $w_2 = 0.1$, $\theta = 0.2$:
  - $(0,0)$: $0 < 0.2 \Rightarrow 0$
  - $(0,1)$: $0.1 < 0.2 \Rightarrow 0$
  - $(1,0)$: $0.1 < 0.2 \Rightarrow 0$
  - $(1,1)$: $0.2 \geq 0.2 \Rightarrow 1$
- Sonuç AND tablosuyla aynıdır. Öğrenme tamamlanmıştır.

Örnek 2: OR öğrenen perceptron (hocanın tahtada çözdüğü örnek)

- Hoca önce "OR yapacağım" dedi, sonra bir öğrencinin uyarısıyla tablonun AND olduğu fark edildi; sonunda OR tablosunu kullandı.
- OR tablosu: $(0,0) \to 0$, $(0,1) \to 1$, $(1,0) \to 1$, $(1,1) \to 1$.
- Başlangıç: $w_1 = 0.2$, $w_2 = 0.2$, $\theta = 0.3$ (sabit), $\alpha = 0.3$, step fonksiyonu.
- 1. epoch:
  - $(0,0)$: $0 < 0.3 \Rightarrow y = 0$, $e = 0$.
  - $(0,1)$: $0.2 < 0.3 \Rightarrow y = 0$, $y_d = 1$, $e = 1$. $w_1 = 0.2 + 0.3 \cdot 1 \cdot 0 = 0.2$, $w_2 = 0.2 + 0.3 \cdot 1 \cdot 1 = 0.5$.
  - $(1,0)$: $0.2 < 0.3 \Rightarrow y = 0$, $e = 1$. $w_1 = 0.2 + 0.3 = 0.5$, $w_2 = 0.5$ kalır.
  - $(1,1)$: $0.5 + 0.5 = 1 > 0.3 \Rightarrow y = 1$, $e = 0$. Değişmez.
- 2. epoch ($w_1 = w_2 = 0.5$, $\theta = 0.3$):
  - $(0,0)$: $0 < 0.3 \Rightarrow 0$
  - $(0,1)$: $0.5 > 0.3 \Rightarrow 1$
  - $(1,0)$: $0.5 > 0.3 \Rightarrow 1$
  - $(1,1)$: $1 > 0.3 \Rightarrow 1$
- Bu epoch'ta hiç hata yoktur. Ağ OR'u öğrenmiştir.

Epoch ile iterasyon farkı (öğrenci sorusu)

- 30 eğitim örneğiniz varsa 30 örneğin hepsini bir kez göstermek bir epoch'tur.
- Tek bir örneği gösterip ağırlıkları güncellemek bir iterasyondur ($t = 3$, $t = 4$ gibi).
- Tek bir örnekte hata çıkmaması öğrenmenin bittiği anlamına gelmez. Tüm örneklerde hata kalmamalıdır.

Sınıflandırma ile regresyon farkı (öğrenci sorusu)

- İkisinde de öğrenme ağırlıkların güncellenmesiyle olur ve gradient descent kullanılır.
- Sınıflandırmada çıkış bir eşikten geçirilir ve sınıf etiketi üretilir ($+1/-1$ veya $1/0$).
- Regresyonda gerçek değerli bir çıkış üretilir; eşik (aktivasyon) uygulanmaz.
- Öğrenmek, karar sınırının nereye konacağına karar vermektir. Destek vektör makinelerinde (SVM) "iyi" ve "daha iyi" karar sınırı farkı görülecek.

6. XOR Problemi

- XOR tablosu: $(0,0) \to 0$, $(0,1) \to 1$, $(1,0) \to 1$, $(1,1) \to 0$.
- AND ve OR'da iki sınıf tek bir doğruyla ayrılabilir. XOR'da ayrılamaz.
- Matematiksel çelişki:
  - $(0,1)$ için $w_2 > \theta$ olmalı.
  - $(1,0)$ için $w_1 > \theta$ olmalı.
  - $(1,1)$ için $w_1 + w_2 < \theta$ olmalı.
  - $(0,0)$ için $0 < \theta$, yani $\theta > 0$.

- Ağırlıklar tek tek eşikten büyükken toplamları eşikten küçük olamaz (eşik pozitifken). Bu yüzden tek katmanlı perceptron XOR'u çözemez.
- Çözüm: doğrusal ayrılamayan problemler için çok katmanlı perceptron.

7. Çok Katmanlı Perceptron ve Backpropagation

- Yapı: giriş katmanı (input layer), gizli katman (hidden layer), çıkış katmanı (output layer).
- Giriş katmanındaki her düğüm gizli katmandaki her düğüme bağlıdır. Gizli katmandaki her düğüm de her çıkış düğümüne bağlıdır.
- XOR için örnek yapı: x_1, x_2 girişleri; 2 gizli nöron (eşikleri θ_1, θ_2); 1 çıkış nöronu (eşiği θ_3; gizliden çıkışa ağırlıklar w_3, w_4).
- Ağırlık sayısı çok olduğu için uzun yıllar yalnız bir gizli katman kullanıldı. Derin öğrenmede gizli katman sayısı arttı ("derinlik").
- MLP doğrusal olmayan bir durum olduğu için aktivasyon olarak sigmoid kullanır:

$$y = \frac{1}{1 + e^{-(\sum_{i=1}^{n} x_i w_i - \theta)}}$$

Backpropagation adımları

1. **Başlatma:** tüm ağırlıkları ve eşikleri küçük rastgele değerlerle başlat. Slayttaki örnekte aralık $\left(-\frac{2.4}{F_i}, +\frac{2.4}{F_i}\right)$, $F_i = i$. nörona gelen giriş sayısı.
2. **İleri yönde aktivasyon:** önce gizli katman, sonra çıkış katmanı çıkışlarını hesapla. Gizli katmanın çıkışları çıkış katmanının girişidir.
3. **Ağırlık eğitimi (geri yönde):**

- Çıkış katmanı hata gradyanı (error gradient):

$$\delta_k(p) = y_k(p)\ [1 - y_k(p)]\ e_k(p), \qquad e_k(p) = y_{d,k}(p) - y_k(p)$$

- Çıkışa giden ağırlıkların düzeltmesi (y_j gizli nöron çıkışı, yani çıkış nöronunun girişi):

$$\Delta w_{jk}(p) = \alpha\, y_j(p)\, \delta_k(p)$$

- Gizli katman hata gradyanı (çıkış katmanından gelen hatalarla hesaplanır, l = çıkış nöronu sayısı):

$$\delta_j(p) = y_j(p)\ \left[1 - y_j(p)\right] \sum_{k=1}^{l} \delta_k(p)\, w_{jk}(p)$$

- Girişten gizli katmana giden ağırlıkların düzeltmesi:

$$\Delta w_{ij}(p) = \alpha\, x_i(p)\, \delta_j(p)$$

4. **İterasyon:** hata belirlenen eşiğin altına inene kadar tekrarla.

- Slayttaki XOR örneğinde toplam kare hata (sum of squared errors) yaklaşık 224 epoch'ta 10^{-3}'ün altına iniyor.
- Gizli nöronların her biri bir karar sınırı çizer. Çıkış nöronu bu iki sınırı birleştirerek XOR bölgesini oluşturur.
- Sigmoid çıkışı 0 veya 1 değil, kesirli bir sayıdır. Bu yüzden hata tam sıfır olmaz; belli bir eşikte durulur.

Çıkış nöronu sayısı (sınıf içi soru)

- 0-9 rakamlarını tanımak için 10 çıkış nöronu kullanılabilir; her rakam için bir nöron 1, diğerleri 0 olur.
- İkili kodlama ile 4 nöron yeterlidir: $0000 \to 0$, $0001 \to 1$, ...

8. Öğrenmeyi Hızlandırma

- Sigmoid yerine hiperbolik tanjant (hyperbolic tangent) kullanılabilir: $y = \dfrac{2a}{1 + e^{-bx}} - a$, a ve b sabit.
- Momentum terimi eklenebilir (genelleştirilmiş delta kuralı):

$$\Delta w_{jk}(p) = \beta\, \Delta w_{jk}(p-1) + \alpha\, y_j(p)\, \delta_k(p)$$

- β 0 ile 1 arasındadır, genelde 0.95 civarında alınır (hoca "0,9 falan" dedi). Momentum ile daha büyük α (ör. 0,5) kullanılabilir.
- Slayttaki örnekte momentum eklenince hata, yaklaşık 224 yerine 126 epoch'ta 10^{-3}'e iniyor (hoca "120 epoch civarı" dedi).

Ders sonrası notlar

- **Duyurular:**
  - Önümüzdeki hafta (24 Kasım 2020) vize haftası. Hoca vizeyi bir sonraki haftaya ertelemeyi planlıyor. Nasıl yapılacağı (kameralar açık vb.) önümüzdeki hafta konuşulacak.
  - Bir öğrenci dönem başında tek sınav olacağının söylendiğini hatırlattı. Hoca yine de vize yapmak istediğini söyledi; ders bir vize, bir final ve ödevlerle yürüyecek.
  - İlk ödev bu hafta Online Yıldız üzerinden verilecek. Öğrenciler ödevi oraya yükleyecek.
  - Bu dersin slaytları (XOR backpropagation örneği dahil) bu hafta Online Yıldız'da paylaşılacak.
- **Düzeltmeler:**
  - Hoca Δw formülünü bir yerde "hesaplanan çıkış çarpı (gerçek çıkış eksi hesaplanan çıkış)" diye söyledi. Doğrusu: $\Delta w_i = \alpha\, x_i\, (y_d - y)$; çarpan hesaplanan çıkış değil, giriş x_i'dir. Hoca birkaç saniye sonra x_i ile doğru söyledi.
  - AND örneğinde hoca w_2'yi önce 0.1 yazdı; slayttaki doğru değer -0.1'dir. Hoca da düzeltti ve öğrencilerden hatalarını uyarmalarını istedi.
  - OR örneğinde ilk yazılan tablo AND idi; bir öğrenci uyarınca OR'a çevrildi.
  - Bir öğrenci Excel'de AND perceptronunu kurdu ve karar doğrusunu $0.1x + 0.1y = 0$ buldu. Hoca eşiğin de hesaba katılması gerektiğini hatırlattı: doğru $0.1x_1 + 0.1x_2 - 0.2 = 0$'dır.
  - Backpropagation formülünde çıkış gradyanındaki çarpan $y_k(1 - y_k)$'dir (sigmoid türevi). Hoca "1 eksi y" kısmını sözlü olarak eksik okudu.
- **Ders düzeni:** Ders 10:40 ve 11:55 civarında iki arayla yapıldı. Ekran paylaşımı birkaç kez koptu; kopmalar içerik kaybına yol açmadı. Gizli katman formülünün anlatıldığı 02:22-02:24 arası altyazıda kopuk; formüller slayttaki standart gösterimden verildi.
- **Kayıt:** Hoca kayıtların sonradan izlenebilmesini avantaj olarak görüyor; itirazı yok.

  Not: Bu özeti ve dersin altyazı dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.